# CGT 390 Lab 9 API and Data Design Report

Mahina Yasutake | Gatherly | October 22, 2026

**Live prototype:** cgt-390-project-seven.vercel.app

**Repository:** github.com/MahinaYasutake/CGT390_Project

## 1. Interaction and contract

The interaction is saving Clay Club to a shortlist and removing it later. Gatherly is a static prototype. Saving has no API endpoint, Server Function, account, or database.

### Entry point

A button with data-save="clay-club" calls the click handler in app.js, lines 115-130. The handler calls toggleSaved(id), writes a list of IDs to localStorage, then updates the interface. On reload, getSaved() reads the same key. These are browser functions.

### Annotated flow

Event Details button → toggleSaved(event ID) → validate ID against the six-event catalog → localStorage key gatherly-saved-events → Saved Events view. The event array supplies public event details; the saved list contains only references to them.

### Contract

Input: a string ID for an existing event. Rule: each ID must exist in the catalog and appear once in the saved list. A second click removes it. Success returns ok:true, the normalized IDs, the selected ID, and saved:true or false. Invalid input or a failed storage write returns ok:false with an error. No save HTTP status exists.

## 2. Data and access decisions

The event ID links a saved-list entry to a fictional catalog record. The JavaScript event array is the catalog source of truth; localStorage is the shortlist store for one browser profile and origin. It is not an account record. The list is limited to valid, unique event IDs; malformed stored data displays as empty until a later valid write.

All event details are public in app.js. The shortlist stores no email, token, or payment field. Anonymous visitors may save; anyone using the same browser profile and origin can use that shortlist. Different origins have separate storage, but there is no user authorization or server validation. Lab 8 account-backed saves and database constraints remain a proposal.

## 3. Testing evidence

Tests used fictional records in isolated Chrome browser contexts on October 22. Identity for each case was an anonymous visitor. Results are function returns, page behavior, and raw localStorage values; save has no separate HTTP response. The complete predicted and actual results are in the before and after test logs.

### T1 Successful action

**Input and prediction:** Save clay-club, reload, then remove and reload. Expect one saved event followed by none. **Actual:** storage was ["clay-club"] with one saved card, then [] with zero cards. Reload confirmed both writes persisted. Passed before and after the repair.

### T2 Invalid input

**Input and prediction:** Change data-save to not-real and an empty ID; expect rejection without a write. **Actual before:** storage became ["not-real",""]. **After:** storage stayed null and the page returned "This event is not available to save." Validation in toggleSaved explains the change; retest passed.

### T3 Access boundary

**Input and prediction:** Save at port 4189, visit the same app at port 4191, then return. Expect the second origin empty and the first unchanged. **Actual:** the second origin had no saved card; the first retained ["clay-club"]. Passed. This checks origin separation, not account ownership.

### T8 Task specific edge case

**Input and prediction:** Open event.html?id=not-real. Expect "Event not found," no Save button, and unchanged storage. **Actual before:** the page substituted Night Market. **After:** it showed "Event not found," no Save button, and storage remained null. The static page request returned HTTP 200; the application displayed the missing-record result.

## 4. Findings and response

The baseline passed 2 of 8 checks. The six failing cases involved invalid IDs, malformed or wrongly shaped storage, duplicates, storage write failure, and unknown detail links. I added ID and list validation, safe parsing, duplicate removal, visible write-failure feedback, and a missing-event state in commit 0a6d62a. All eight affected and unaffected checks passed after repair, plus a direct function validation check.

Selected evidence: invalid IDs changed storage from ["not-real",""] before repair to null after repair; a malformed value {broken caused a page error before repair but rendered with count 0 after repair. Successful saves were checked after reload; rejected writes were checked for unchanged storage. Full logs and selected screenshots record the remaining cases.

The merged prototype also restores the documented Lab 7 mobile and deferred analytics behavior. Eleven local responsive and analytics checks and three live task flows passed. The Lab 6 accessibility repairs were verified in the live prototype, including result announcements and the Save button state.

Remaining limits: no account authorization, database persistence, reservation, or concurrent-write protection. External analytics delivery was not tested. These are outside the current browser-local contract.

## 5. Peer review and AI use

### Peer review

Pending. No classmate has yet proposed and run an additional case, so there is no peer result to report. Share the contract above and ask a classmate to choose a new case, such as two tabs changing the shortlist. Record their input, prediction, actual result, and any repair or blocker before submission.

### AI use

ChatGPT/Codex reviewed the code, built and ran the tests, implemented repairs, and drafted this report. The claims above were checked against the code, browser results, and stored values. I reviewed the document, decided on which action to investigate, and drafted parts of this report.